Cooking Assistant

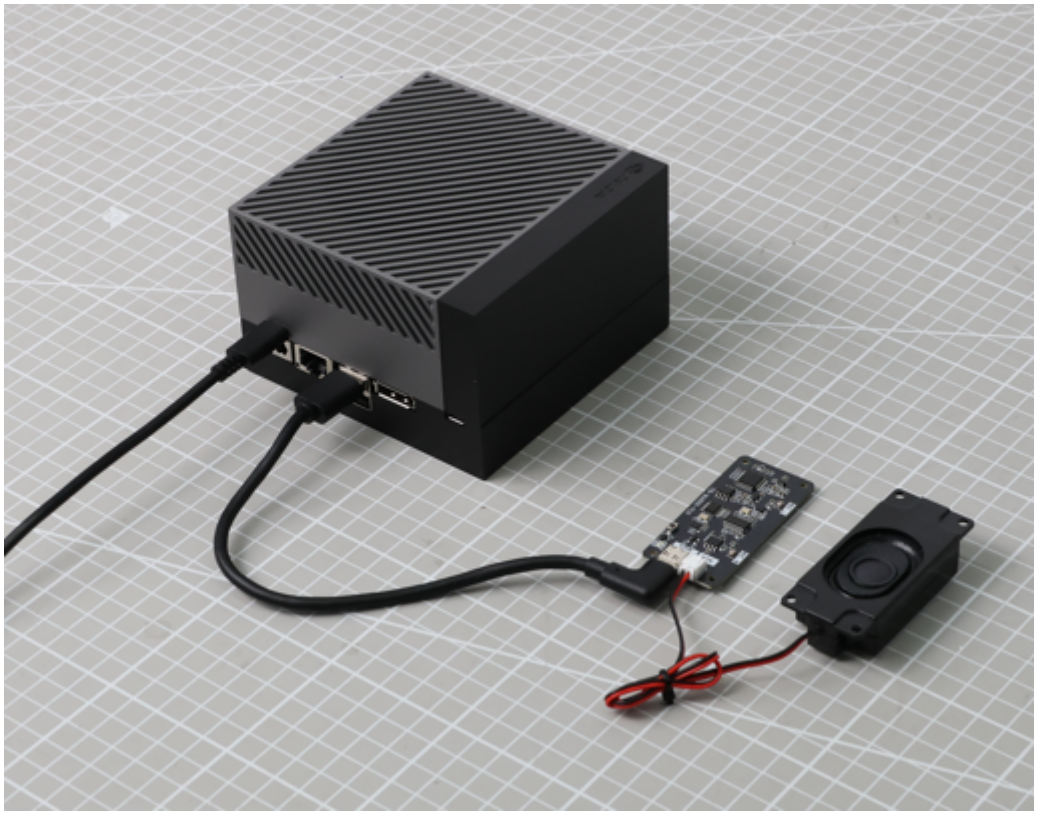
Cooking Assistant
1. Function Introduction
2. Hardware Connection and Setup
3. Code Analysis
4. Running the Code
5. Experimental Results

1. Function Introduction

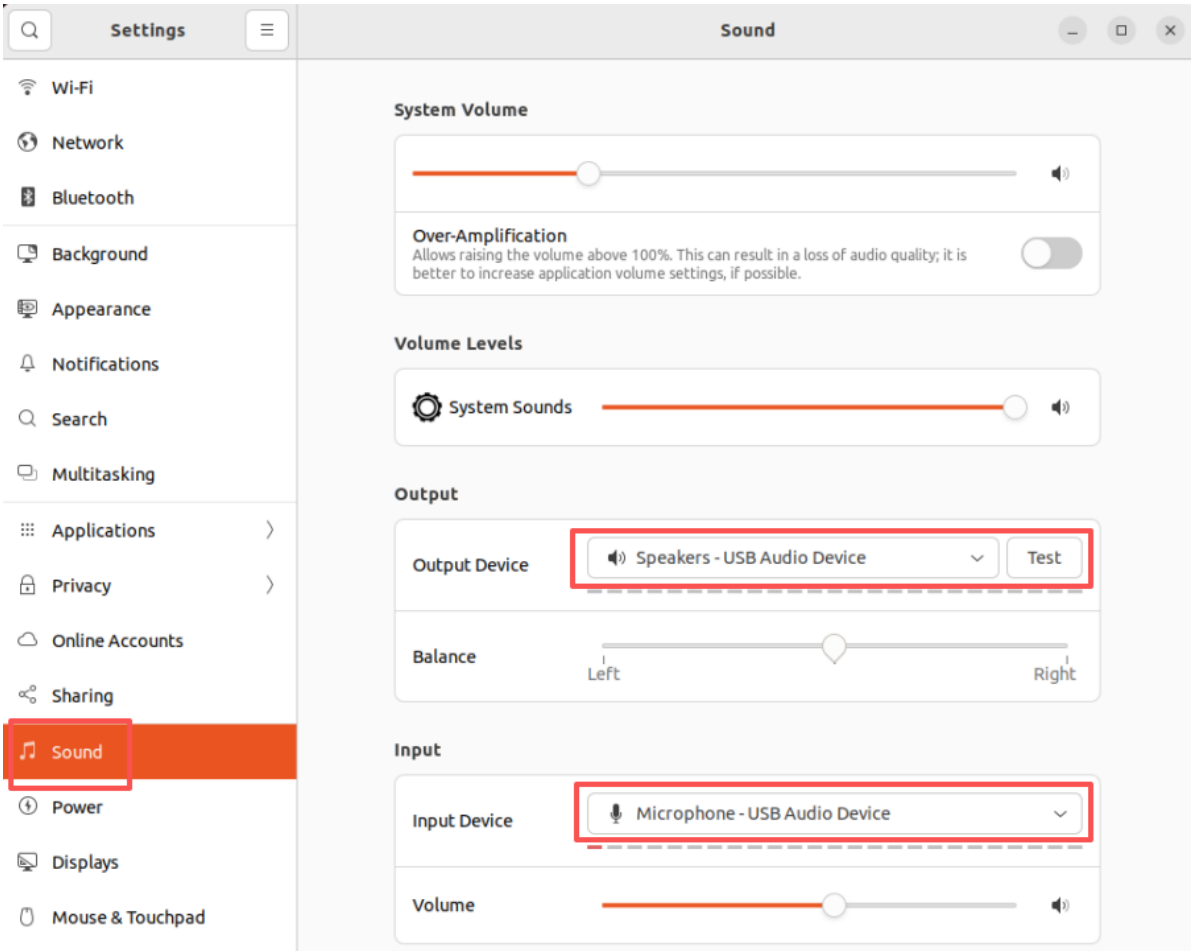
This example can generate recipes for projects, including ingredient preparation, seasoning amounts, cooking methods, and the final result image.

2. Hardware Connection and Setup

This example requires not only a power adapter but also a large language model module for language input and output, a DP monitor, and a mouse and keyboard for displaying the interface and operations.



Open system settings, click Sound, select Speakers-USB Audio Device for Output Device, and Microphone-USB Audio Device for Input Device.



3. Code Analysis

Source Code Path:

```
/home/jetson/showing/cooking_assistant/cooking_assistant.py
```

To adjust the image window size, modify the values of self.win_width and self.win_height.

```
self.win_name = "Storyteller"
# self.win_width = 1920
# self.win_height = 1080
self.win_width = 1024
self.win_height = 768
self._window_ready = False
```

To modify the recording duration, modify the value of the -d parameter, which indicates how many seconds of recording will automatically stop; the default is 10 seconds.

```python
audio_file = TMP_DIR / f"recording_{int(time.time())}.wav"
cmd = [
    "arecord",
    # "-D", self.capture_devstr or "default",
    "-f", self.capture_format,
    "-r", str(self.capture_rate),
    "-c", str(self.capture_channels),
    "-d", "10",
    str(audio_file),
]
print(f"[AUDIO] Recording command: {' '.join(cmd)}")
process = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
```

If you need to force text input, remove the comment symbol from `text` and write the custom text.

```python
text = " ".join(s.text.strip() for s in segments).strip()
# text = "There is a bird flying in the sky."
# text = "天空中有一个鸟"
print(f"[AUDIO] Transcribed: '{text}'")

if text:
    self.start_story_generation(text)
else:
    print("[AUDIO] No speech detected")
```

If you need to modify the start prompt, modify the parameter passed to `generate_tts`.

```python
def play_opening_audio(self):
    """Play or create opening audio"""
    opening_audio = TMP_DIR / "opening.wav"

    if not opening_audio.exists():
        print("[AUDIO] Creating opening audio...")
        self.generate_tts("大佬，请稍等。 美食菜谱马上给您呈上来。", opening_audio)
        self._trim_wav_tail(opening_audio, tail_ms=150)
    if opening_audio.exists():
        print("[AUDIO] Playing opening audio...")
        self.play_audio_file(opening_audio)
```

If you need to modify the content of the generated story and images, you can modify the `story_prompt` variable.

```python
def generate_func_thread(self, user_prompt):
    """Generate func using Ollama streaming API"""
    func_prompt = f"""你现在是我的私人厨师，你富有创造力和独特性，你需要在我接下来话中提取我想要的菜谱，每个菜谱都是根据我的个人口味
定制的。请用中文回答。
你的特色:
    你使用简单的词汇和简短、清晰的句子结构。
    你熟悉中国美食，并且熟悉制作方法。
    你分享的菜谱尽量不重复。
我的个人口味:
我是一个中国人，喜欢中式美食。
我喜欢清淡，不喜欢太咸，不喜欢吃辣。
我对虾和蟹过敏，其他海鲜可以推荐。
特殊指令:
你拥有一项特殊能力。在讲解菜谱完毕前，你必须插入一个图像标签。
    [IMAGE: 整道美食的详细描述]: 此描述必须是为最终美食的完整且详细的提示，需描述美食的主食、配料、颜色、味道等信息。
接下来我的话是:
"{user_prompt}" """
```

If you need to modify the serial port of the large language model module, the port of ComfyUI, the Ollam server port, or the piper model file, modify the parameters below.

```python
def main():
    parser = argparse.ArgumentParser(description="Interactive Storyteller Application")
    parser.add_argument('--serial-port', '-s', default='/dev/ttyUSB0',
                        help='Serial port for device')
    parser.add_argument('--comfyui-server', '-c', default='127.0.0.1:8188',
                        help='ComfyUI server address')
    parser.add_argument('--ollama-server', '-o', default='http://localhost:11434',
                        help='Ollama server URL')
    parser.add_argument('--piper-model', '-p', default=None,
                        help='Piper TTS model path')
    args = parser.parse_args()
```

4. Running the Code

Open a new system terminal and start the ComfyUI service.

```
cd ~/ComfyUI
source venv/bin/activate
python main.py
```

```
jetson@ubuntu:~$ cd ~/ComfyUI
jetson@ubuntu:~/ComfyUI$ source venv/bin/activate
(venv) jetson@ubuntu:~/ComfyUI$ python main.py
```

```
Import times for custom nodes:
   0.0 seconds: /home/jetson/ComfyUI/custom_nodes/websocket_image_save.py

Context impl SQLiteImpl.
Will assume non-transactional DDL.
Assets scan(roots=['models']) completed in 0.348s (created=0, skipped_existing=2
2, orphans_pruned=0, total_seen=22)
Starting server

To see the GUI go to: http://127.0.0.1:8188
```

Open a new system terminal, start the virtual environment, and launch the main storytelling program.

```
cd ~/showing/
source venvs/bin/activate
cd cooking_assistant
python cooking_assistant.py
```

```
jetson@ubuntu:~$ cd ~/showing/
jetson@ubuntu:~/showing$ source venvs/bin/activate
(venvs) jetson@ubuntu:~/showing$ cd cooking_assistant
(venvs) jetson@ubuntu:~/showing/cooking_assistant$ python cooking_assistant.py
```

5. Experimental Results

After the program starts.

Say "Hi,yahboom" to the large language model module to wake it up, then say the dish you want to eat. Wait for the recording to finish, and the speaker will play a prompt to prepare the recipe. Then, the recipe will be generated, converted into speech, and played. Finally, a picture of the food will be displayed.

To end playback, you can say "Hi, yahboom" to the large language model module to interrupt and restart recording and recipe generation.